SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 1: Foundations of Programming and the Modern Agricultural Computing Environment

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

This document serves as the primary study material for Module 1 of AGE 219. It is designed to bridge the gap between traditional engineering mechanics and modern digital agriculture (AgriTech).

Contents

1 Introduction: The Engineering Need for Computation **3**
1.1 The Evolution of Agricultural Technology . . . 3
1.2 What is a Computer Program? . . . 3

2 Understanding the Machine: Computer Architecture for Engineers **4**
2.1 The Central Processing Unit (CPU) . . . 4
2.2 Random Access Memory (RAM) . . . 4
2.3 Storage (Hard Drive / SSD) . . . 4
2.4 Input/Output Devices (Peripherals) . . . 4

3 Algorithmic Thinking and the IPO Model **5**
3.1 The IPO Model . . . 5
3.2 Pseudocode . . . 5

4 Why Python for Agricultural Engineering? **6**
4.1 1. Readability and Syntax . . . 6
4.2 2. The "Batteries Included" Philosophy and Ecosystem . . . 6
4.3 3. Relevance to AgriTech . . . 6

5 The Development Environment: Google Colaboratory **7**
5.1 What is Google Colab? . . . 7
5.2 Key Advantages of Colab . . . 7
5.3 The Anatomy of a Colab Notebook . . . 7
5.3.1 1. Text Cells (Markdown) . . . 7
5.3.2 2. Code Cells . . . 7
5.3.3 Executing a Cell . . . 7
5.3.4 The Kernel (Runtime) . . . 8

6 Hands-on Session: The "Hello, Farm" Protocol and Beyond **8**
6.1 Step 1: Initializing the Environment . . . 8
6.2 Step 2: The Print Function . . . 8
6.3 Step 3: Variables and Memory Allocation . . . 9
6.4 Step 4: Basic Arithmetic and Operators . . . 9

7 Case Studies in Agricultural Engineering **10**
7.1 Case Study 1: Irrigation Pumping Requirements . . . 10
7.2 Case Study 2: Tractor Field Capacity . . . 10

8 Introduction to Debugging **11**
8.1 1. Syntax Errors . . . 11
8.2 2. Runtime Errors (Exceptions) . . . 11
8.3 3. Logical Errors (The Most Dangerous) . . . 12

9 Summary **12**

10 Glossary of Terms **13**

11 Extended Self-Assessment Exercises **14**
11.1 Exercise 1: The Personal Profile . . . 14
11.2 Exercise 2: Engine Displacement Calculation . . . 14
11.3 Exercise 3: Financial Crop Yield Projection . . . 14

11.4 Exercise 4: Debugging Practice 14
11.5 Exercise 5: Markdown Mastery 15

1 Introduction: The Engineering Need for Computation

Welcome to AGE 219. As students of Agricultural, Irrigation, and Post-Harvest Engineering, you are entering the profession at a pivotal moment in human history. Traditionally, agricultural engineering was heavily focused on mechanical, civil, and thermodynamic systems: designing stronger plows, calculating the flow rate in concrete canals, or engineering the thermal dynamics of a grain dryer. While these physical laws remain the foundation of our profession, they are no longer sufficient on their own.

We are currently in the era of **Agriculture 4.0** (or Digital Agriculture). The modern engineering landscape is defined by the *Digital Twin* concept—the ability to create a digital, mathematical replica of a physical farm system. Programming is the critical bridge that allows us to translate physical laws into digital models that can predict, automate, and optimize farm systems in real-time.

1.1 The Evolution of Agricultural Technology

To understand why programming is essential, we must look at the evolution of agricultural technology:

- **Agriculture 1.0 (Pre-20th Century):** Characterized by high labor, animal power, and simple mechanical hand tools.
- **Agriculture 2.0 (The Green Revolution - 1950s):** The advent of the combustion engine, synthetic fertilizers, and mechanical tractors. Engineering was purely mechanical and chemical.
- **Agriculture 3.0 (Precision Agriculture - 1990s):** The integration of GPS signals into tractors. This allowed for auto-steering and the beginnings of variable-rate technology.
- **Agriculture 4.0 (The Present & Future):** Highly autonomous systems. Drones mapping crop health using multispectral cameras, soil sensors connected to the internet (IoT) automatically triggering irrigation valves, and machine learning algorithms predicting yield based on decades of weather data.

In Agriculture 4.0, the hardware (the tractor, the pump, the silo) is directed by software. If you cannot understand, write, or modify that software, your ability to engineer modern agricultural solutions is severely limited.

1.2 What is a Computer Program?

At its most fundamental level, a computer program is a sequence of unambiguous instructions that a computer follows to achieve a specific result.

Humans are highly intuitive. If you tell a farm worker, ”Water the tomatoes if the soil looks dry,” the worker uses visual processing, tactile feedback (touching the soil), and past experience to execute the task perfectly.

Computers, however, have absolutely no intuition. A computer is simply a very fast calculator made of sand (silicon) and metal. It requires explicit, mathematical logic. To instruct a computer to water the tomatoes, an engineer must write a program that specifies:

1. **Input:** Which sensor pin is the soil moisture probe connected to? How often should we read it? (e.g., Read Sensor A1 every 60 seconds).
2. **Processing:** What is the mathematical threshold for ”dry”? (e.g., If moisture voltage is less than 2.1V, or $moisture < 30\%$).

3. **Output:** Which relay controls the water pump? How long should it remain open? (e.g., Send 5V to Relay B for 600 seconds).

2 Understanding the Machine: Computer Architecture for Engineers

Before we write code, we must understand the physical system we are controlling. Just as you must understand the components of an internal combustion engine before you can optimize its fuel efficiency, you must understand computer architecture before you can write efficient code.

Let us use an analogy comparing a computer to a modern agricultural milling facility.

2.1 The Central Processing Unit (CPU)

The CPU is the ”brain” of the computer, or the **Facility Manager** in our analogy. The CPU performs all the mathematical calculations and logical decisions. It processes the instructions you write in your Python code. However, the CPU can only do one or a few things at a time, albeit incredibly fast (billions of calculations per second).

2.2 Random Access Memory (RAM)

RAM is the short-term working memory of the computer. In our analogy, it is the **Processing Floor or Workbench**. When you start a program, the data and instructions are loaded from the hard drive into the RAM because the CPU can access the RAM much faster than the hard drive.

- If you are analyzing a massive dataset of 10 years of rainfall data for the Morogoro region, all that data must be loaded into RAM.
- If your dataset is larger than your available RAM (e.g., trying to load 16GB of satellite imagery on an 8GB RAM laptop), the computer will ”crash” or slow to a halt. As engineers, we must write code that uses memory efficiently.

2.3 Storage (Hard Drive / SSD)

Storage is the long-term memory. This is the **Grain Silo** or **Warehouse**. Data stored here is permanent even when the power is turned off (unlike RAM, which is wiped clean when the computer shuts down). Your Python `.py` files, CSV datasets, and exported graphs live here.

2.4 Input/Output Devices (Peripherals)

For an agricultural computer (like an Arduino, Raspberry Pi, or a tractor’s Electronic Control Unit), the Input/Output (I/O) devices are the connection to the physical world.

- **Inputs (Sensors):** Thermocouples (temperature), anemometers (wind speed), load cells (weight/yield monitors).
- **Outputs (Actuators):** Solenoid valves (irrigation), stepper motors (robotic arms), variable frequency drives (ventilation fans).

Your Python code acts as the intermediary, taking signals from the Inputs, processing them in the CPU using RAM, and sending commands to the Outputs.

3 Algorithmic Thinking and the IPO Model

Programming is not simply about learning a language like Python; it is about learning how to think algorithmically. An **algorithm** is a finite sequence of well-defined, computer-implementable instructions to solve a class of problems or perform a computation.

3.1 The IPO Model

Most engineering programs follow the **IPO Model**: Input, Process, Output. Let us look at a classic agricultural engineering problem: calculating the Equilibrium Moisture Content (EMC) of grain to determine if a drying fan should be turned on.

1. Input

What data does the system need to solve the problem?

- Ambient Air Temperature (T) from a sensor.
- Ambient Relative Humidity (RH) from a sensor.
- Grain Type constants (since maize dries differently than rice).

2. Process

What equations and logic must be applied to the inputs? We might use the Modified Henderson Equation for EMC:

$$EMC = \left[\frac{-\ln(1 - RH)}{K \cdot (T + C)} \right]^{\frac{1}{N}} \tag{1}$$

Where $K, C,$ and N are crop-specific constants. The algorithm calculates this math, and then makes a logical decision: *Is the calculated EMC lower than the current grain moisture? If yes, the air is dry enough to dry the grain.*

3. Output

What is the final result of the program?

- Send a signal to turn the ventilation fan **ON**.
- Print a message to the farmer's dashboard: "Drying in progress. Current EMC: 12%."

3.2 Pseudocode

Before typing actual Python code, engineers often write "Pseudocode." Pseudocode is a plain-English outline of the algorithm. It allows you to design the logic without worrying about missing commas or brackets.

Example Pseudocode: Automated Irrigation

```
START
READ soil_moisture_level from Sensor 1
READ weather_forecast for "Rain Probability"

IF soil_moisture_level < 25% AND Rain_Probability < 50%:
        TURN ON water_pump
        WAIT for 30 minutes
        TURN OFF water_pump
ELSE:
        KEEP water_pump OFF

LOG action to database
END
```

4 Why Python for Agricultural Engineering?

There are hundreds of programming languages (C++, Java, Rust, MATLAB). In this course, we will exclusively use **Python**. Python was created by Guido van Rossum in 1991 and has since become the dominant language in science, engineering, and data analysis.

4.1 1. Readability and Syntax

Python was designed to be easily readable. It uses English keywords instead of complex punctuation. It relies on *indentation* (whitespace) to structure code, rather than the curly braces used in C++. This means you spend less time memorizing syntax rules and more time solving engineering problems.

4.2 2. The "Batteries Included" Philosophy and Ecosystem

Python has a massive ecosystem of pre-written code collections called **Libraries** or **Modules**. As an engineer, you rarely have to write complex math algorithms from scratch.

- **NumPy:** Used for heavy numerical computations and matrix algebra (vital for structural engineering and fluid dynamics).
- **Pandas:** Used for analyzing massive datasets, like 10 years of tractor GPS coordinates or crop yield maps.
- **Matplotlib:** Used for creating publication-quality engineering graphs and charts.
- **SciPy:** Contains modules for optimization, linear algebra, integration, and differential equations.

4.3 3. Relevance to AgriTech

Python is the industry standard for Artificial Intelligence and Machine Learning (via libraries like TensorFlow and PyTorch). If you want to train a drone to recognize diseased maize leaves using computer vision, you will use Python. If you want to analyze GIS (Geographic Information System) shapefiles for a farm topography, you will use Python.

5 The Development Environment: Google Colaboratory

Traditionally, to write Python, you had to install the Python Interpreter on your computer, configure system path variables, and download an Integrated Development Environment (IDE) like PyCharm or VS Code. This often led to compatibility issues depending on whether a student had Windows, Mac, or Linux, or if they had an older, slower laptop.

To eliminate these barriers, we will use **Google Colaboratory (Colab)**.

5.1 What is Google Colab?

Google Colab is a free, cloud-based programming environment hosted by Google. It is based on the open-source Jupyter Notebook project. Instead of running code on your physical laptop's CPU and RAM, Colab sends your code over the internet to a powerful Google server in a data center. The Google server executes the code and sends the results back to your browser.

5.2 Key Advantages of Colab

1. **Zero Configuration:** All you need is a web browser (Chrome, Firefox, etc.) and a Google account.
2. **Cloud Computing Power:** Even if you have a 10-year-old laptop, you can write code that utilizes massive processing power, including Google's GPUs (Graphics Processing Units) for free.
3. **Collaboration:** Just like Google Docs, you can share your Colab notebook with your classmates or instructor. We can edit the code together in real-time.
4. **Integration:** It saves automatically to your Google Drive.

5.3 The Anatomy of a Colab Notebook

A Colab Notebook is an interactive document made up of blocks called **Cells**. There are two main types of cells:

5.3.1 1. Text Cells (Markdown)

Text cells are used for documentation. Good engineering code must be documented. In Colab, text cells use a formatting language called **Markdown**. Markdown allows you to easily create bold text, bulleted lists, and even complex mathematical formulas.

For example, to write an equation in a text cell, you enclose LaTeX math code in dollar signs: `$ A = \pi r^2 $`. When you execute the cell, it beautifully renders the math.

5.3.2 2. Code Cells

Code cells are where you type your actual Python commands. They have a gray background and a "Play" button (a right-facing triangle) on the left side.

5.3.3 Executing a Cell

To make the computer read and execute the code in a cell, you can:

- Click the "Play" button on the left of the cell.

- Or, use the keyboard shortcut: **Shift + Enter** (This runs the current cell and moves the cursor to the next cell—highly recommended for speed).

5.3.4 The Kernel (Runtime)

When you open a Colab notebook, Google assigns a virtual computer to you. This connection is called the "Kernel" or "Runtime." Look at the top right of the Colab screen; you will see a button that says "Connect" or displays RAM and Disk usage graphs. If your notebook sits idle for too long, Google will disconnect your runtime to save server resources, and you will have to reconnect and run your cells again from the top.

6 Hands-on Session: The "Hello, Farm" Protocol and Beyond

It is time to write your first lines of code. Please follow these steps carefully.

6.1 Step 1: Initializing the Environment

1. Open your web browser.
2. Ensure you are logged into your Google Account.
3. Navigate to the URL: `https://colab.research.google.com`
4. A welcome window will appear. Click on **New Notebook** at the bottom right of the window.
5. A new, blank notebook will open. Click on the title at the top left (it usually says `Untitled0.ipynb`) and rename it to `AGE_219_Lecture1_YourName.ipynb`. Note: The `.ipynb` extension stands for Interactive Python Notebook.

6.2 Step 2: The Print Function

The most basic command in Python is the `print()` function. It tells the computer to display information to the screen.

Click on the first empty Code Cell and type the following exactly as written:

```
# This is my first Python program for AGE 219
print("Hello, Farm!")
print("Digital Agriculture Initialization Complete.")
```

Explanation:

- **Line 1:** The `#` symbol indicates a **Comment**. The computer completely ignores any text after the `#` on that line. Comments are for humans to read. Use them to explain *why* your code does what it does.
- **Lines 2 & 3:** `print()` is a built-in Python function. The text inside the parentheses must be enclosed in quotation marks (`"` or `'`). Text enclosed in quotation marks is called a **String** (a string of characters).

Now, press **Shift + Enter**. The cell will execute, and you will see the output printed below the cell.

6.3 Step 3: Variables and Memory Allocation

In engineering calculations, we constantly deal with changing values: temperature, flow rate, RPM, area. In programming, we store these values in **Variables**.

A variable is like a named box in the computer's RAM. You give the box a name, and you put a value inside it.

Create a new Code Cell and enter the following:

```
# Defining variables for a tractor
tractor_brand = "Massey Ferguson"
engine_horsepower = 120
is_4wd = True

# Printing the variables
print("Tractor Details:")
print(tractor_brand)
print(engine_horsepower)
```

Rules for Naming Variables:

- Names must start with a letter or an underscore _.
- Names cannot contain spaces. Use underscores to separate words (e.g., `soil_moisture`, not `soil moisture`). This is called *snake_case*.
- Names are case-sensitive. `Yield` and `yield` are two different variables.
- Choose descriptive names. Do not use `x = 120`. Use `engine_horsepower = 120`. Descriptive names make your code readable.

6.4 Step 4: Basic Arithmetic and Operators

Python acts as a highly advanced calculator. It uses standard mathematical operators:

- Addition: `+`
- Subtraction: –
- Multiplication: `*`
- Division: `/`
- Exponentiation (Power): `**` (e.g., 3^2 is written as `3**2`)

Let us write a program to calculate the volume of a cylindrical grain silo. We know the formula is $Volume = \pi \times r^2 \times h$.

Create a new Code Cell:

```
# Grain Silo Volume Calculator
pi_value = 3.14159
radius_meters = 4.5
height_meters = 15.0

# Calculate the volume using the formula
volume_cubic_meters = pi_value * (radius_meters ** 2) * height_meters

# Output the result
print("The calculated volume of the silo is:")
print(volume_cubic_meters)
print("cubic meters.")
```

When you run this cell, Python automatically retrieves the values stored in `pi_value`, `radius_meters`, and `height_meters`, substitutes them into the mathematical formula, calculates the result, and stores it in the new variable `volume_cubic_meters`. Finally, it prints the result.

7 Case Studies in Agricultural Engineering

To solidify these concepts, let us look at two practical engineering problems and how we solve them using basic Python programming.

7.1 Case Study 1: Irrigation Pumping Requirements

Problem Statement: An irrigation engineer needs to calculate the total volume of water required to irrigate a specific farm and the time it will take a specific pump to deliver that water.

Given Parameters:

- Farm Area: 2.5 Hectares
- Required Application Depth: 30 mm
- Pump Flow Rate: 15 Liters per second (L/s)

Engineering Logic (Pseudocode): 1. Convert Area from Hectares to square meters ($1Ha = 10,000m^2$). 2. Convert Application Depth from mm to meters ($1mm = 0.001m$). 3. Calculate Total Volume required in cubic meters ($Volume = Area \times Depth$). 4. Convert Volume from cubic meters to Liters ($1m^3 = 1000L$). 5. Calculate Time required in seconds ($Time = TotalLiters / PumpFlowRate$). 6. Convert Time to hours for practical understanding.

Python Implementation: Type this into a new Code Cell in Colab:

```
# --- Inputs ---
area_ha = 2.5
depth_mm = 30.0
pump_flow_L_per_sec = 15.0

# --- Process (Conversions and Calculations) ---
# 1 Ha = 10000 m^2
area_m2 = area_ha * 10000

# 1 mm = 0.001 m
depth_m = depth_mm / 1000

# Volume = Area * Depth
volume_m3 = area_m2 * depth_m

# 1 m^3 = 1000 Liters
total_liters = volume_m3 * 1000

# Time = Volume / Flow Rate
time_seconds = total_liters / pump_flow_L_per_sec

# Convert seconds to hours (3600 seconds in an hour)
time_hours = time_seconds / 3600

# --- Output ---
print("Total Water Required (Liters):", total_liters)
print("Time required to irrigate (Hours):", time_hours)
```

Execute the code. You will see how Python cleanly handles a multi-step engineering conversion process. By simply changing the input variables at the top, you can instantly recalculate for a different farm or a different pump size.

7.2 Case Study 2: Tractor Field Capacity

Problem Statement: Calculate the Theoretical Field Capacity (TFC) of an agricultural implement in Hectares per Hour (ha/hr).

Formula:

$$TFC = \frac{S \times W}{10}$$

Where:

- S = Speed of the tractor in km/hr
- W = Width of the implement in meters
- 10 = A constant conversion factor

Python Implementation:

```
# Inputs for a Disc Harrow operation
tractor_speed_kmh = 8.5
implement_width_m = 3.2

# Calculate Theoretical Field Capacity
tfc_ha_per_hr = (tractor_speed_kmh * implement_width_m) / 10

# Output
print("The Theoretical Field Capacity is:")
print(tfc_ha_per_hr)
print("Hectares per Hour")
```

8 Introduction to Debugging

As an engineer, you know that physical machines break. Software breaks too. When a program fails to execute properly, it generates an **Error**. The process of finding and fixing these errors is called **Debugging**.

There are three main types of errors in programming:

8.1 1. Syntax Errors

This occurs when you violate the grammatical rules of the Python language. It is like writing a sentence in English without a verb. Python will refuse to run the code at all.

Example of a Syntax Error:

```
# Missing the closing quotation mark
print("Hello Farm)
```

Colab Output: `SyntaxError: unterminated string literal`

8.2 2. Runtime Errors (Exceptions)

The code's grammar is correct, but something goes wrong while the program is actively running. A common runtime error in engineering is dividing by zero.

Example of a Runtime Error:

```
# Attempting to divide by zero
pump_flow = 0
time = 100 / pump_flow
```

Colab Output: `ZeroDivisionError: division by zero`

Another very common runtime error is a **NameError**, which happens if you misspell a variable name:

```
my_yield = 50
# Misspelled the variable in the print statement
print(my_yeild)
```

Colab Output: `NameError: name 'my_yeild' is not defined`

8.3 3. Logical Errors (The Most Dangerous)

The code runs perfectly. There are no error messages. However, the result is mathematically incorrect because the engineer formulated the logic or equation wrong.

Example of a Logic Error:

```
# Calculating the average of two moisture readings
reading1 = 15
reading2 = 25

# INCORRECT LOGIC due to order of operations
average = reading1 + reading2 / 2
print(average)
```

Output: `27.5` (The correct answer is 20. Python divided `reading2` by 2 first, then added `reading1`. The engineer forgot to use parentheses: `(reading1 + reading2) / 2`).

Debugging Tip: Always manually calculate your first test case on a piece of paper or a calculator to verify that your program's logic is sound!

9 Summary

In this extensive foundational lecture, we have covered:

- The critical transition to **Agriculture 4.0** and why programming is non-negotiable for modern agricultural engineers.
- The basic architecture of a computer (CPU, RAM, Storage, I/O) and how it maps to an agricultural system.
- The **IPO Model** (Input, Process, Output) for algorithmic thinking.
- The advantages of the **Python** language and the **Google Colab** environment.
- Writing introductory Python code, including `print()` statements, variable assignments, and mathematical operations.
- Solving practical engineering problems (Irrigation Volume, Field Capacity) using code.
- Identifying basic programming errors (Syntax, Runtime, Logic).

In Lecture 2, we will dive deeper into **Data Types** (understanding the strict difference between numbers and text) and explore more advanced operators to handle complex datasets.

10 Glossary of Terms

- **Algorithm:** A set of step-by-step instructions designed to solve a specific problem.
- **Bug:** An error, flaw, or fault in a computer program that causes it to produce an incorrect or unexpected result.
- **Colab (Google Colaboratory):** A free cloud-based Jupyter notebook environment that requires no setup and runs entirely in the browser.
- **Comment:** Text in a program that is ignored by the compiler/interpreter, used to explain the code to human readers. In Python, it begins with #.
- **CPU (Central Processing Unit):** The primary component of a computer that acts as its ”brain,” performing mathematical and logical operations.
- **IDE (Integrated Development Environment):** A software application that provides comprehensive facilities to computer programmers for software development (e.g., PyCharm, VS Code).
- **IoT (Internet of Things):** A network of physical objects (like soil sensors or weather stations) embedded with sensors and software for the purpose of connecting and exchanging data over the internet.
- **Kernel:** The computational engine executing the code contained in a Notebook document.
- **Markdown:** A lightweight markup language that you can use to add formatting elements (bold, lists, headers) to plaintext text cells in Colab.
- **RAM (Random Access Memory):** The short-term, volatile memory of a computer where currently running programs and data are stored.
- **String:** A data type used in programming to represent text rather than numbers. Denoted by quotation marks in Python.
- **Syntax:** The set of rules that defines the combinations of symbols that are considered to be correctly structured programs in a specific language.
- **Variable:** A named storage location in the computer’s memory used to hold data that can be modified during program execution.

11 Extended Self-Assessment Exercises

To master programming, you must practice writing code yourself. Reading the lecture is only the first step. Open your Google Colab notebook and complete the following exercises.

11.1 Exercise 1: The Personal Profile

Create a new code cell. Create four variables to store your personal information: 1. `first_name` 2. `last_name` 3. `degree_program` (e.g., "BSc Agricultural Engineering") 4. `graduation_year` Use multiple `print()` statements to display a neat profile card.

11.2 Exercise 2: Engine Displacement Calculation

Agricultural engineers frequently work with engines. Write a program to calculate the total engine displacement (volume) of a 4-cylinder tractor engine. The formula for the volume of a single cylinder is $V = \pi \times r^2 \times stroke$. **Given parameters to input into your variables:**

- Bore (Diameter of the cylinder) = 100 mm (Note: you must calculate radius r from this).
- Stroke length = 120 mm.
- Number of cylinders = 4.

Goal: Your program must calculate the total displacement in cubic centimeters (cm^3) and print the result. (Hint: convert mm to cm first).

11.3 Exercise 3: Financial Crop Yield Projection

A farmer wants to project their revenue. Write a program that takes the following inputs:

- Total farm area in acres = 50
- Expected yield per acre = 1.8 tons
- Current market price per ton of maize = 650,000 TZS

Calculate the total expected yield in tons, and the total projected revenue in Tanzanian Shillings (TZS). Print both results clearly with descriptive text.

11.4 Exercise 4: Debugging Practice

Copy the following broken code into a Colab cell. Run it, observe the error message, identify the problem, and fix the code so it runs correctly.

```
# Broken Code for calculating bulk density
dry_soil_mass = 120
total_soil_volume = 100

bulk_density = dry_soil_mass / total_soil_volume

print("The bulk density is:")
print(bulk_dens)
```

11.5 Exercise 5: Markdown Mastery

Create a **Text Cell** in your notebook. Use Markdown formatting to create: 1. A large Header that says ”My Engineering Goals”. 2. A bulleted list of 3 things you hope to learn in AGE 219. 3. Bold the most important goal.

End of Lecture 1 Compendium